

NLP & PyTorch Fundamentals

সহজ ভাষায় ডিপ লার্নিং ও প্রাকৃতিক ভাষা প্রক্রিয়াকরণ (NLP) সম্পর্কিত প্রয়োজনের পূর্ণাঙ্গ গাইড

পার্ট ১: ডেটা প্রিপারেশন ও ভোকাবুলারি

Q1 Dataset কী এবং কেন এটি মডেল ট্রেনিংয়ের আগে প্রয়োজন?

Dataset কী: Dataset হলো একটি নির্দিষ্ট বিষয়ে সুন্দরভাবে গোছানো তথ্যের (Data) সংগ্রহ। এতে টেক্সট, ছবি, অডিও বা সংখ্যার মতো বিভিন্ন ধরনের তথ্য থাকতে পারে।

কেন প্রয়োজন: মানুষ যেমন বাস্তব উদাহরণ দেখে শেখে, Machine Learning বা Deep Learning মডেলও ডেটাসেটের প্যাটার্ন (Patterns) বিশ্লেষণ করে শেখে। ট্রেনিংয়ের পূর্বে ডেটাসেট না দিলে মডেল কিসের ভিত্তিতে সিদ্ধান্ত নেবে তা বুঝতে পারে না।

Q2 Tokenization কী এবং বাক্যকে কেন ছোট টোকেনে ভাঙা হয়?

Tokenization: একটি পুরো বাক্য বা প্যারাগ্রাফকে ছোট ছোট অংশ বা একক শব্দ/সাব-ওয়ার্ডে ভাগ করার প্রক্রিয়াকে Tokenization বলা হয়।

কেন ভাঙা হয়: Neural Network বা কম্পিউটার আস্ত বাক্য একবারে প্রসেস করতে পারে না। প্রতিটি টোকেনকে আলাদা করে বুঝতে এবং সেগুলোকে গাণিতিক ভেক্টরে রূপান্তর করতে বাক্য ভেঙে ছোট টোকেন করা জরুরি।

Q3 Lowercasing না করলে কী সমস্যা হতে পারে?

যদি "What" এবং "what"-কে আলাদা শব্দ ধরা হয়, তবে দুটি প্রধান সমস্যা দেখা দেয়:

- Vocabulary-র আকার বৃদ্ধি:** একই শব্দের ভিন্ন কেসিংয়ের জন্য একাধিক Index তৈরি হয়, যা মেমোরি অপচয় করে।
- প্যাটার্ন শেখার ক্ষতি:** মডেল মনে করবে "What" (বাক্যের শুরুতে) এবং "what" (বাক্যের মাঝে) দুটি সম্পূর্ণ ভিন্ন জিনিস, যা মডেলের শেখার দক্ষতা কমায়।

Q4 Vocabulary কী এবং এটি কীভাবে Integer-এ Map হয়?

Vocabulary: একটি NLP প্রজেক্ট বা মডেল যেসব অনন্য (Unique) শব্দ চিনতে ও বুঝতে পারে, সেগুলোর তালিকাকে Vocabulary বলে।

```
# Vocabulary Mapping Example vocab = {"<UNK>": 0, "what": 1, "is": 2, "capital": 3}
```

এখানে শব্দগুলোকে গাণিতিক সূচকে (Numerical Index) রূপান্তর করা হয় যাতে কম্পিউটার সহজে প্রসেস করতে পারে।

Q5 <UNK> (Unknown) Token কী এবং Vocabulary-তে শব্দ না থাকলে কী হয়?

কী হয়: যদি কোনো শব্দ Vocabulary-তে না থাকে (Out-of-Vocabulary / OOV), তবে মডেল সরাসরি ত্রুটি (Error) দেবে অথবা বাক্যটি বুঝতে ব্যর্থ হবে।

<UNK>-এর ভূমিকা: এই সমস্যা সমাধানের জন্য অচেনা শব্দগুলোর জায়গায় <UNK> টোকেন বসানো হয়, ফলে মডেল না আটকে পড়া চালিয়ে যেতে পারে।

Q6 "What is the capital of France" সরাসরি PyTorch Model-এ দেওয়া যায় না কেন?

Neural Network এবং PyTorch কেবল গাণিতিক হিসাব বা **Numerical Tensor** (Matrix/Vector of Numbers) বোঝে। সরাসরি অ্যালফাবেট বা স্ট্রিং (Text String) গাণিতিক অপারেশনে অংশ নিতে পারে না। তাই টেক্সটকে প্রথমে Tokenizer দিয়ে Index-এ এবং তারপর Tensor-এ রূপান্তর করে মডেলে দিতে হয়।

Q7 Dataset vs DataLoader, batch_size এবং shuffle-এর গুরুত্ব

Dataset vs DataLoader

Dataset: সব ডেটা লোড করে এবং ১টি করে Sample (`__getitem__`) দেয়।

DataLoader: ব্যাচ তৈরি, ডেটা এলোমেলো (Shuffle) করা ও ইটারেট করা সহজ করে।

batch_size vs shuffle

batch_size: একসাথে কয়টি স্যাম্পল মডেলে যাবে তা ঠিক করে (GPU মেমোরি ঠিক রাখে)।

shuffle: প্রতি Epoch-এ ডেটার ক্রম এলোমেলো করে ওভারফিটিং রোধ করে।

Q8 nn.Embedding() কী কাজ করে এবং Random Integer-এর চেয়ে এটি কেন সেরা?

কাজ: `nn.Embedding()` প্রতিটি Discrete Integer Index-কে একটি Dense Continuous Vector-এ রূপান্তর করে।

কেন সেরা: Random Integer (যেমন: what=1, apple=45) দিয়ে শব্দের অর্থগত সম্পর্ক (Semantic Relationship) প্রকাশ করা যায় না। কিন্তু Vector Embedding-এ সম্পর্কিত শব্দগুলোর (যেমন: "king" ও "queen") ভেক্টর গাণিতিক স্পেসে কাছাকাছি অবস্থান করে।

Q9 PyTorch-এ squeeze() এবং unsqueeze()-এর পার্থক্য ও ব্যবহার

পার্থক্য: `unsqueeze(dim)` নির্দিষ্ট ডাইমেনশনে একটি নতুন 1-আকারের ডাইমেনশন যুক্ত করে এবং `squeeze(dim)` 1-আকারের থাকা ডাইমেনশনকে মুছে দেয়।

RNN-এ ভূমিকা: RNN/LSTM ইনপুটে Batch Dimension প্রত্যাশা করে। Single Sequence প্রসেস করার সময় Batch Dim (1) যোগ করতে `unsqueeze(0)` ব্যবহৃত হয়।

```
import torch x = torch.zeros(10, 32) # Shape: [10, 32] -> (seq_len, feature_dim) x
```

Q10 RNN কীভাবে Sequence প্রসেস করে এবং Hidden State কী?

RNN একটি Sequence-এর শব্দগুলো একে একে (Step-by-Step) প্রসেস করে।

Hidden State (\$h_t\$): এটি হলো RNN-এর মেমোরি। t সময়ের শব্দ x_t প্রসেস করার সময় এটি আগের ধাপের Hidden State h_{t-1} -কে গ্রহণ করে নতুন স্মৃতি h_t তৈরি করে, যা পরবর্তী শব্দে প্রভাব ফেলে।

Q11 nn.CrossEntropyLoss() কী মেজার করে এবং এটি কেন উপযুক্ত?

কী মেজার করে: এটি মডেলের দেওয়া Probability Distribution এবং আসল Target Class-এর মধ্যে পার্থক্য বা ভুল পরিমাপ করে (Log-Softmax ও NLLLoss-এর সংমিশ্রণে)।

কেন উপযুক্ত: এটি Multi-class Classification-এর জন্য সেরা। ভোকাবুলারির হাজার হাজার শব্দের মধ্য থেকে সঠিক শব্দটি বেছে নিতে এটি মডেলকে সঠিক সম্ভাবনা নির্দেশ করতে বাধ্য করে।

Q12 Adam Optimizer এবং loss.backward() এর কাজ

Optimizer-এর কাজ: loss.backward()-এর পর optimizer.step() কল করলে Optimizer গ্র্যাডিয়েন্ট অনুযায়ী মডেলের প্যারামিটার আপডেট করে ($W_{\text{new}} = W_{\text{old}} - \eta \cdot \nabla L$)।

বারবার আপডেটের কারণ: একবারে সঠিক Weight পাওয়া অসম্ভব। বারবার ছোট ছোট পদক্ষেপে প্যারামিটার আপডেট করে লস কমিয়ে সঠিকতায় পৌঁছাতে হয়।

Q13 Answer-এর প্রথম Token-কে Target ধরা নিয়ে পর্যালোচনা

সমস্যা: শুধু প্রথম টোকেনকে টার্গেট ধরলে মডেল কেবল উত্তরের প্রথম শব্দটিই শিখবে, পুরো উত্তর তৈরি করতে পারবে না।

সঠিক পদ্ধতি: Sequence-to-Sequence Loss ব্যবহার করতে হবে। প্রতিটি Time Step-এ পর্যায়ক্রমিক টোকেন প্রেডিক্ট করিয়ে সবগুলোর লস যোগ করে ব্যাকপ্রোপাগেশন করতে হবে।

Q14 Question-Answering-এর জন্য সঠিক Architecture (Seq2Seq)

সাধারণ ১-টোকেন RNN QA-এর জন্য যথেষ্ট নয়। এর জন্য প্রয়োজন **Encoder-Decoder (Seq2Seq)** আর্কিটেকচার:

- **Encoder:** প্রশ্ন প্রসেস করে একটি Context Vector তৈরি করে।
- **Decoder:** Context Vector নিয়ে $\langle \text{EOS} \rangle$ না পাওয়া পর্যন্ত অটোরিগ্রেসিভ উপায়ে উত্তর তৈরি করে।
- **Teacher Forcing:** ট্রেনিং দ্রুত করার জন্য ডিকোডারে আসল বিগত টোকেন ইনপুট দেওয়া হয়।

Q15 Vocabulary তৈরিতে Data Leakage এবং সঠিক নিয়ম

কেন সমস্যা (Data Leakage): সম্পূর্ণ ডেটাসেট (Test set সহ) ব্যবহার করে Vocabulary তৈরি করলে Test Set-এর অজানা শব্দগুলো আগেই প্রকাশ পেয়ে যায়, ফলে ইভালুয়েশন ভুল বা Biased হয়।

সঠিক নিয়ম: Vocabulary তৈরি করতে হবে শুধুমাত্র **Training Set** থেকে। Test বা Validation সেট-এ অচেনা শব্দ পাওয়া গেলে সেগুলোকে সরাসরি $\langle \text{UNK} \rangle$ দিয়ে প্রতিস্থাপন করতে হবে।